

AstroSpec

A Modular Desktop Suite for High-Resolution
Spectropolarimetric Analysis

Prepared By:

Riyan Alvi

Project Architecture & Systems Report

Indian Institute of Science Education & Research Mohali

August 2026

TABLE OF CONTENTS

Abstract	3
1 Introduction & Project Motivation	3
1.1 Project Scientific Purpose	3
1.2 Problem Statement & Project Genesis	3
1.3 Comparative Analysis of Existing Systems	3
2 Core Data Domain & Object Hierarchy	4
2.1 Structural Object Taxonomy	4
2.2 Atomic Channel Component (Spectrum)	4
2.3 Integration Multi-Channel Frame (Observation)	4
2.4 Global Target Tracker (AstroObject)	4
3 Desktop GUI Architecture & Interactive Event Tracking	5
3.1 Component Layout Matrix (AstroSpecWindow)	5
3.2 Canvas Event Intercepts and Hover Callbacks	5
3.3 Permanent Pin Aggregations and Overlay Caches	6
4 Algorithmic Processing & Vector Mathematics	6
4.1 Automated 3rd-Order Continuum Estimation	6
4.2 Background Removal via 8-Pass Iterative σ -Clipping	6
4.3 Linear Polarimetric Vector Extraction	6
5 Bootstrap Lifecycle & System Entry	6
5.1 Frameless Interface Initialization	6
5.2 Dynamic Environment Path Resolution	7
5.3 Non-Blocking Application Handoff	7
6 System Integration & Ingestion Routing	7
6.1 The Controller Signal Architecture (PlotManager)	7
6.2 The Operational Traffic Router (PlotDataRouter)	7
7 Conclusions & Future Extensions	7
7.1 Engineering Summary	7
7.2 Future Structural Frameworks	7
References	8

Abstract

This report details the architectural design and algorithmic execution of **AstroSpec**, a modular, desktop-based spectropolarimetric data analysis and visualization suite. Developed in Python using an event-driven **PyQt5** user interface and a hardware-accelerated **Matplotlib** canvas backend, the application converts raw, multi-extension Flexible Image Transport System (FITS) data logs into interactive workspaces. The software applies a strict Model-View-Controller (MVC) architecture to run heavy input/output (I/O) streams and matrix calculations without blocking user interface responsiveness. Key processing modules include world coordinate system (WCS) wavelength calibrations, automated 3rd-order polynomial continuum estimations, background profile removal via an 8-pass iterative σ -clipping regression filter, and spatial vector transformations for extracting linear polarization profiles. Furthermore, the framework integrates low-level canvas event hooks to connect live cursor movement and mouse-clicks to an interactive data tracking spreadsheet for point interrogation. Backed by defensive array dimension validation guards and metadata search fallbacks, AstroSpec provides a stable, responsive desktop tool for verifying scientific data configurations, prepping spectrum baselines, and accelerating astrophysics data assessment workloads.

1 Introduction & Project Motivation

1.1 Project Scientific Purpose

In observational astrophysics, parsing the physical state of remote cosmic objects depends on decomposing high-resolution light profiles. Raw celestial light captured by ground-based or space-borne observatories carries overlapping structural patterns: a bright blackbody radiation background (the continuum) superimposed with distinct atomic absorption dips or emission spikes.

Additionally, tracking cosmic dust structures or interstellar magnetic fields requires analyzing polarimetric vectors to evaluate directional shifts across light fields. The **AstroSpec** application provides an integrated, cross-platform desktop suite designed to automate these baseline steps, ingest raw data archives, process vector mathematics, and render interactive charts within a single, unified workspace.

1.2 Problem Statement & Project Genesis

The development of AstroSpec was directly motivated by the operational inefficiencies of traditional command-line astronomical data handling. Originally, inspecting high-performance scientific archives required writing multiple disconnected Python scripts: first, separate script loops had to be engineered to extract text records and isolate target properties from raw data structures; next, secondary search routines were required to scan

data columns for correct calibration constants; finally, entirely separate scripts had to be written to structure and plot the resulting data vectors.

This manual scripting model presented significant challenges:

- **High Workflow Overhead:** Researchers had to modify and run separate script configurations just to check data quality or verify file headers.
- **Disjointed Analysis Environments:** Separating data lookup scripts from plotting routines made it tedious to inspect individual coordinates on the fly.
- **Interface Thread Locking:** Standard plotting scripts frequently froze system windows when calculating multi-pass matrix arrays.

AstroSpec removes these bottlenecks by consolidating file ingestion, keyword discovery, wavelength calibration, and visualization into a single desktop tool. This architecture gives researchers access to powerful array processors through an intuitive point-and-click interface.

1.3 Comparative Analysis of Existing Systems

To define the operational scope of AstroSpec, its performance and accessibility must be evaluated against existing astronomical reduction frameworks. While institutional tools are available for FITS processing, they exhibit distinct structural gaps:

1. **Legacy Software Incompatibilities (IRAF / PyRAF):** The Image Reduction and Analysis Facility (IRAF) remains an industry standard but suffers from severe software bitrot. Developed in the mid-1980s, native installation on modern 64-bit operating systems is restricted, often requiring complex virtual machines or legacy containers. AstroSpec resolves this by utilizing native Python 3 architectures.
2. **Disjointed Scripting Frameworks (Astropy Ecosystem):** Modern Python packages like Astropy provide robust standalone data-handling utilities. However, they function purely as programmatic libraries rather than end-user applications. Researchers are forced to write fragmented scripts to extract header content, verify calibration coefficients, and plot results. AstroSpec abstracts this by integrating these libraries beneath an event-driven GUI.
3. **Commercial and Non-Specific Packages (MATLAB / AstroImageJ):** Commercial graphing utilities offer point inspection tools but require paid proprietary licensing and lack native FITS file processing layers. Conversely, open-source astronomical visualization apps are heavily optimized for spatial imaging and photometry rather than intensive spectropolarimetric data workflows.

Table 1 provides a feature matrix contrasting AstroSpec against these alternate systems.

2 Core Data Domain & Object Hierarchy

2.1 Structural Object Taxonomy

To eliminate the runtime instabilities of passing unmapped arrays and loose dictionaries between detached scripts, AstroSpec abstracts raw telescope variables into a unified object hierarchy. The system isolates data states across three distinct boundaries: Target Parameters, Integration Sessions, and Physical Data Channels.

2.2 Atomic Channel Component (Spectrum)

The `Spectrum` class forms the base data element of the domain hierarchy. It instantiates a self-contained tracking module that binds file matrices

directly to physical coordinates, avoiding the need for loose variable matching at runtime:

- **`self.flux`:** The raw data vector representing intensity metrics extracted from the primary HDU extension table.
- **`self.wavelength`:** Calibrated physical coordinates computed in Angstrom units (Å) using linear WCS scaling.
- **`self.error`:** A matching matrix representing empirical data uncertainty, retrieved through dynamic keyword searches.
- **`self.spectrum_type`:** A categorical string identifier used by routing layers to assign vectors to polarization bins (STOKES_I, STOKES_Q, or STOKES_U).

2.3 Integration Multi-Channel Frame (Observation)

The `Observation` class aggregates independent spectral components into a single telescope runtime framework. This component unifies separate file streams into a specific structural channel layout:

```
self.spectra = {
    "STOKES_I": None, # Total Flux Intensity Profile
    "STOKES_Q": None, # Linear Polarization ( Horizontal vs. Vertical Axis)
    "STOKES_U": None, # Linear Polarization ( Diagonal Axis Translation)
    "ERROR": None # Shared Fallback Uncertainty Vector Array
}
```

This consolidation enables the routing architecture to merge separate file extractions into a single observation, making multi-channel calculations straightforward.

2.4 Global Target Tracker (AstroObject)

The highest tier in the data structure, `AstroObject`, functions as a reference ledger for specific astronomical coordinate targets. It manages an internal chronological tracking stack (`self.observations = []`) that dynamically links multiple separate observation records to a single object name. This setup simplifies time-series tracking and longitudinal source variations across multi-epoch monitoring programs.

Table 1: Functional Feature Matrix Across Scientific Reduction Frameworks

Architectural Feature	IRAF / PyRAF	Pure Astropy Scripts	AstroImageJ	AstroSpec
Native FITS Ingestion	Yes	Yes	Yes	Yes
Zero-Coding Interface	No	No	Yes	Yes
Automated WCS Mapping	Yes	No (Manual Scripting)	Yes	Yes
Stokes Polarimetry Tools	Complex	No (Manual Scripting)	No	Yes
Iterative σ -Clipping	Complex	No (Manual Scripting)	No	Yes
Modern OS Compatibility	Poor	Excellent	Excellent	Excellent

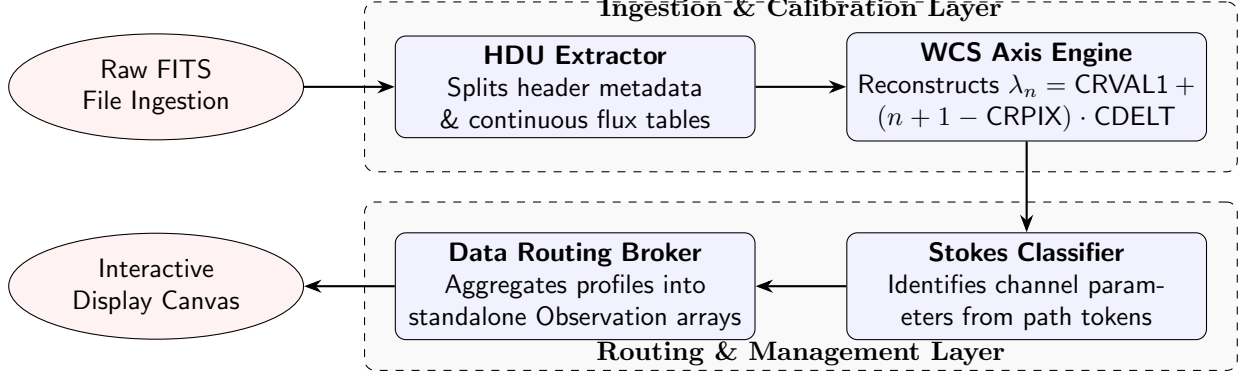


Figure 1: End-to-End Scientific Data Pipeline Flowchart: Processing from Raw Binary FITS Files to the Hardware-Accelerated Graphical Presentation Layer.

3 Desktop GUI Architecture & Interactive Event Tracking

3.1 Component Layout Matrix (AstroSpecWindow)

The desktop view organizes workspace components using precise dimensional layout ratios to optimize visual presentation. The main application window partitions screen real estate into a coordinated, multi-panel dashboard where responsive user controls are balanced against a wide center plotting block:

- **SideBar (Width Factor 1):** Manages file-tree collections, database tree indices, and active dataset selection flags using a tree structure (`QTreeWidget`) configured with checking filters (`Qt.ItemIsUserCheckable`).
- **SpectrumPlot (Width Factor 6):** The central visualization workspace, running a hardware-accelerated, dark-themed Matplotlib canvas instance wrapped within a PyQt5 user interface framework.
- **AnalysisPanel (Width Factor 2):** Tracks metrics, cursor coordinates, and system parameter logs via an interactive data spreadsheet container (`QTableWidget`) displaying multi-column coordinates.

3.2 Canvas Event Intercepts and Hover Callbacks

To let researchers manually interrogate data lines, `SpectrumPlot` binds low-level mouse movements and click inputs directly to internal callback routers using Matplotlib's native connection manager:

```

self.canvas.mpl_connect("motion_notify_event",
    self.mouse_move)
self.canvas.mpl_connect("button_press_event",
    self.mouse_click)
    
```

The hover intercept listener (`mouse_move`) detects continuous movements on active axes, processing coordinate positions through precision floating-point text formatters inside the `AnalysisPanel`: The hover intercept listener (`mouse_move`) detects continuous movements on active axes, processing coordinate positions through precision floating-point text formatters inside the `AnalysisPanel`:

$$\lambda_{\text{readout}} \rightarrow \{\text{wavelength:.3f}\} \text{ \AA} \quad (1)$$

$$Flux_{\text{readout}} \rightarrow \{\text{flux:.5e}\} \quad (2)$$

This event loop updates the UI instantly, tracking coordinates smoothly underneath the mouse without delaying background data calculations.

This event loop updates the UI instantly, tracking coordinates smoothly underneath the mouse without delaying background data calculations.

3.3 Permanent Pin Aggregations and Overlay Caches

When an operator registers a left mouse click over a data point, the tracking loop (`mouse_click`) executes a multi-tier annotation deployment pipeline:

1. **Coordinate Capture:** Grabs raw numerical data values from the plot axes and records them inside an tracking array (`self.pinned_points`).
2. **Overlay Drawing:** Places a distinct marker overlay (`marker="o", color="red"`) directly over the clicked canvas coordinate.
3. **Canvas Labeling:** Generates a high-contrast text annotation (`annotate`) numbered based on array length, shifting text markers slightly to remain clear (`xytext=(8,12)`).
4. **PyQt Panel Update:** Emits a localized system signal (`point_pinned.emit(x,y)`), adding a new parameter row directly to the coordinate spreadsheet tracker.

To wipe out user markers when loading a new profile, `clear_pins` cleans up graph layers systematically. It loops through internal artist arrays, running native removal methods (`artist.remove()`) to free up canvas memory without leaving orphaned UI textures.

4 Algorithmic Processing & Vector Mathematics

4.1 Automated 3rd-Order Continuum Estimation

To track system background baselines without processing emission line profiles, the `ContinuumPlot` engine applies an absolute median tolerance window. It computes a centralized median benchmark and rejects extreme variations by mapping a boolean array filtering mask ($\mathbf{M}$):

$$\mathbf{M} = (0.7 \cdot F_{\text{median}} < Flux < 1.3 \cdot F_{\text{median}}) \quad (3)$$

A third-degree least-squares polynomial regression matrix fits the remaining unmasked wavelength data points:

$$P_3(\lambda) = c_3\lambda^3 + c_2\lambda^2 + c_1\lambda + c_0 \quad (4)$$

4.2 Background Removal via 8-Pass Iterative σ -Clipping

To flatten continuous astronomical background slopes, the `ContinuumSubtractedPlot` subsystem runs a multi-pass background estimation loop. For each iteration pass $k \in \{1, \dots, 8\}$, a second-order curve maps the current active coordinates:

$$C_k(\lambda) = \text{polyfit}(\lambda_{\mathbf{M}_k}, Flux_{\mathbf{M}_k}, \text{deg} = 2) \quad (5)$$

The residual distance matrix ($\mathbf{R} = Flux - C_k(\lambda)$) checks the array noise limits. Outlying points are thrown out on subsequent iterations if they cross a strict threshold window:

$$\mathbf{M}_{k+1} = (\mathbf{R} < 1.5 \cdot \text{std}(\mathbf{R}_{\mathbf{M}_k})) \quad (6)$$

Terminal profile flattening subtracts the final calculated trend line ($Flux - C_8(\lambda)$), centering the remaining lines along a baseline at $Y = 0$.

4.3 Linear Polarimetric Vector Extraction

The application derives the linear polarization amplitude profile $P(\lambda)$ from the normalized Stokes parameters $q(\lambda)$ and $u(\lambda)$:

$$P(\lambda) = \sqrt{Q(\lambda)^2 + U(\lambda)^2} \quad (7)$$

Before running this calculation, the module verifies that the dimension spaces match to prevent runtime errors.

5 Bootstrap Lifecycle & System Entry

5.1 Frameless Interface Initialization

The bootstrap panel class (`RobustSplashScreen`) suppresses default operating system window boundaries to display a smooth startup asset animation:

```
self.setWindowFlags(Qt.FramelessWindowHint |
    Qt.WindowStaysOnTopHint)
self.setAttribute(Qt.WA_TranslucentBackground)
```

The layout centers itself across varied monitor shapes by sampling display limits directly from the user's workspace profile: The layout centers itself across varied monitor shapes by sampling display limits directly from the user's workspace profile:

$$\mathbf{C}_{\text{desktop}} = \text{Center}\left(\text{QApplication.desktop()}.availableGeometry()\right) \quad (8)$$



Figure 2: The Model-View-Controller (MVC) Signal Loop: Asynchronous coordination separating user inputs from core array matrix storage.

5.2 Dynamic Environment Path Resolution

To keep path definitions valid when wrapping the code into compressed binaries, a dual-mode asset locator handles path resolution dynamically:

```

if hasattr(sys, '_MEIPASS'):
    base_dir = sys._MEIPASS
else:
    base_dir = os.path.dirname(os.path.abspath(
        __file__))
  
```

5.3 Non-Blocking Application Handoff

An event-driven timing module transitions the application state without locking user interface systems:

```

QTimer.singleShot(3000, self.launch_main_app)
  
```

When the timer fires, the system creates the primary workspace framework (*AstroSpecWindow*), launches the dashboard panels, and terminates the splash screen assets to free up memory.

6 System Integration & Ingestion Routing

6.1 The Controller Signal Architecture (PlotManager)

The integration core relies on an event-driven router block governed by the *PlotManager* class. Rather than hardcoding distinct canvas redraw loops for separate datasets, the system handles view configurations dynamically using native Qt slots. When a user interacts with the top tab row selection system or updates sidebar checkboxes, a unified update thread triggers:

```

# Dynamic layout execution lifecycle hook
self.plot_manager.plot_changed.connect(self.
    update_header_plot)
  
```

6.2 The Operational Traffic Router (PlotDataRouter)

To prevent memory fragmentation caused by copying multi-megabyte array arrays during view

switches, data retrieval is managed by the *PlotDataRouter* data broker. This class receives structural view name string requests from the tab layouts and extracts targeted index layers from the tracking data structures:

```

if plot_name == "raw":
    return observation.spectra["STOKES_I"]
elif plot_name == "polarization":
    return observation # Passes entire object
                        # tracking space for vector magnitude
                        # equations
elif plot_name == "multiple":
    return [spec for spec in observation.
            spectra.values() if spec is not None]
  
```

This approach separates data storage states from canvas drawing code. The drawing canvas receives only the explicit mathematical data arrays needed for the current chart view, keeping the user interface fast and responsive.

7 Conclusions & Future Extensions

7.1 Engineering Summary

The development of *AstroSpec* demonstrates how wrapping complex astronomical calculation libraries inside a clean desktop interface can streamline research workflows. By consolidating file parsing, dynamic keyword mapping, linear WCS wavelength calibration, and iterative background continuum corrections into a single desktop tool, the software eliminates the need for researchers to write and run separate, fragmented plotting scripts. The decoupled Model-View-Controller design pattern manages heavy processing loops smoothly, ensuring the user interface remains responsive while handling large telescope datasets.

7.2 Future Structural Frameworks

While the application successfully automates data ingestion, baseline tracking, and real-time point interaction, future development will expand its core analytical features:

1. **Data Pipeline Generalization:** Modifying the ingestion engine to accept spectroscopic

and polarimetric data formats from a wider array of ground-based and space-borne optical, infrared, and X-ray observatories, moving beyond its current optimization for the Fermi Gamma-ray Space Telescope archive.

2. **Automated Atomic Line Identification:** Integrating a local database of atomic transitions (e.g., hydrogen Balmer series, ionized helium profiles) to automatically overlay chemical markers based on the calibrated wavelength axis.
3. **Gaussian Component Line Fitting:** Developing a mathematical optimization layer using least-squares minimization algorithms to fit Gaussian, Lorentzian, and Voigt profiles directly over emission lines to measure equivalent widths and radial velocities.

References

- [1] The Astropy Collaboration, A. M. Price-Whelan, P. L. Lim, *et al.*, “The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Core Python Package and Ecosystem,” *The Astrophysical Journal*, vol. 935, no. 2, p. 167, 2022.
- [2] W. D. Pence, L. Chiappetti, C. G. Page, *et al.*, “Definition of the Flexible Image Transport System (FITS), version 3.0,” *Astronomy & Astrophysics*, vol. 524, p. A42, 2010.
- [3] E. Collett, *Field Guide to Polarization*. Bellingham, Washington: SPIE Press, 2005.